

Preguntas Proyecto

o ¿Cómo manejaron la herencia y el polimorfismo en su diseño? ¿Qué ventajas les brindó?

Se definió una clase Usuario como clase padre, y de ella heredaron Estudiante e Instructor. Esto permitió reutilizar atributos y métodos comunes (id, nombre, email, registrarse). El polimorfismo se aplicó en métodos como consultarCalificaciones() y registrarCalificacion(), ya que cada tipo de usuario los usa de forma distinta. Ventaja: se redujo la duplicación de código y facilitó la extensibilidad del sistema

o ¿Qué estructuras de datos fueron más útiles y por qué?

Listas: para almacenar estudiantes, cursos, evaluaciones y calificaciones, ya que permiten un manejo dinámico de elementos.

Diccionarios: útiles para relacionar rápidamente códigos de curso con su información. Estas estructuras facilitaron la búsqueda y organización eficiente de datos.

o ¿Qué errores comunes anticiparon y cómo los controlaron en el código?

- **Cursos duplicados** → control con validación de código único.
- **Calificaciones inválidas** → manejo de excepciones para aceptar solo valores numéricos válidos.
- **Usuario inexistente** → verificación de que el id exista antes de inscribir o calificar.
- **Errores de entrada** → uso de try/except para evitar que el programa se detenga por datos incorrectos.

o ¿Cómo organizaron su trabajo usando Git y GitHub?

Por temas de comidad decidimos realizar una sola rama para todo el trabajo completo sin embargo todos trabajamos en la construcción del código y apartir de segundo comit que fue en el que trabajo se empezó a trabajar por partes se deja especificado quien hiso que parte todos trabajamos individual sin embargo alguien se encargo del control de versiones en git con apoyo de los demas

o ¿Qué mejoras harían si tuvieran más tiempo?

- Integrar una **base de datos SQL** en lugar de listas para mayor persistencia.
- Crear una **interfaz gráfica** o aplicación web para mejor usabilidad.

- Añadir **notificaciones automáticas** (ej. alertar a estudiantes con promedio bajo).
 - Implementar **roles avanzados** (ej. coordinadores académicos).
- Reflejar las dificultades encontradas y cómo las resolvieron.
- **Problema:** estructurar las relaciones entre clases (ej. curso-estudiante-evaluación).
Solución: crear clases intermedias como Calificacion que unen entidades.
 - **Problema:** duplicación de código en métodos comunes de usuarios.
Solución: aplicar **herencia** con una clase base Usuario.
 - **Problema:** organización del trabajo en equipo.
Solución: usar **GitHub con ramas** y acuerdos de estilo de código.